02_upstream_feature_inventory

HXC-159 Phase 1a — Exhaustive HelixSkills power-feature inventory

Field	Value
Revision	1
Created	2026-07-29
Last modified	2026-07-29
Status	Complete — 755 exported symbols enumerated, 0 parse errors, completeness proof in §11
Workable item	HXC-159 (Task, Queued, High)
Upstream	`git@github.com:HelixDevelopment/skills.git`
Default branch	`main` @ `315b56ce1e4c2570610e0b214ea34d64fc469e1e` (2026-07-18)
Method	Read-only clone into scratch; `go/ast` full-tree symbol enumeration

---

Table of contents

---

1. Executive summary — what HelixSkills actually is

`HelixDevelopment/skills` contains a **production-grade Go service** — the *HelixKnowledge Skill Graph System*, module `github.com/helixdevelopment/skill-system`, go 1.25.5 — that stores skills as a directed dependency graph in PostgreSQL+pgvector and serves them over REST, MCP, ACP, CLI and TUI, with LLM-driven auto-expansion, evidence-based validation, tree-sitter code analysis, multi-source ingestion, Redis caching, Prometheus metrics and multi-tenancy.

The single most important structural fact for HXC-159: the entire Go implementation is **not** at the repository root. It is buried at `docs/research/mvp/Agent_AI_Skill_Tree_Development/project/` — a *research MVP* path. All 237 Go files, `go.mod`, `migrations/`, `deploy/`, `Makefile` and `scripts/` live under that one directory. The repository root has no Go module at all. §4 covers the consequences.

Scale (measured, §11):

Metric	Value
Go files (non-test + test)	237
Top-level declarations (non-test)	1 418
Exported symbols (non-test)	**755**
Packages with exported surface	26
Exported interfaces (extension points)	13
MCP tools	13
REST route groups / endpoints	4 groups, 12 handlers
CLI command groups	7 (+ root), 27 subcommands
Worker job types	6
DB migrations / tables	12 files / 10 tables
Branches / tags	9 / **0**
Parse errors during enumeration	**0**

---

2. The three-layer architecture (load-bearing)

This is the part most likely to be mis-modelled in later phases, so it is stated explicitly. There are **three different things** in this ecosystem that are all called “skills”, and they live in three different repositories:

```
LAYER 1 — SKILL SOURCE (authoring)          repo: HelixDevelopment/HelixCo
  constitution/skills/<name>/SKILL.md         YAML frontmatter: name, descr
  constitution/skills/<name>/register.sh      symlinks the skill into a con
  constitution/skills/<name>/SKILL.{html,pdf,docx}   §11.4.65 exports
        │
        │  register.sh → constitution/scripts/install_cli_agent_plugins.sh
        │  auto-invoked by constitution/scripts/post_update_hook.sh on eve
        │  constitution pull (§11.4.164)
        ▼
LAYER 2 — SKILL GRAPH SERVICE (storage/serving)   repo: HelixDevelopment/s
  Go service: PostgreSQL+pgvector graph, REST + MCP + ACP + CLI + TUI + wo
  Ingests skills from GitHub / filesystem / URL sources, dedups, validates
  auto-expands via LLM, searches semantically.
        │
        │  internal/skillscatalog.Generate()
        ▼
LAYER 3 — GENERATED CATALOGUE (published output)  repo: HelixDevelopment/s
  docs/skills/skill/<name>.md    "GENERATED FILE — DO NOT HAND-EDIT"
  docs/skills/by-domain/*.md, docs/skills/by-kind/*.md, docs/skills/INDEX.
  docs/skills/.catalog_fingerprint   (§11.4.86 drift-proof fingerprint)
```

Evidence for the Layer-1 → consumer wiring, verbatim from `constitution/skills/action-prefix-system/register.sh`:

```
# Purpose:   Register the 'action-prefix-system' skill in a
        consuming project by
#           linking it into <project>/.claude/skills/ (the
        native project-scoped
#           Agent-Skills discovery path). Called by
        post_update_hook.sh on every
#           constitution pull (§11.4.164) and by
        install_cli_agent_plugins.sh.
# Side-effects: creates <project>/.claude/skills/ and one symlink
        (never a copy —
#           the constitution is inherited BY REFERENCE per
        §11.4.28 / §11.4.80).
```

A generated Layer-3 file states the same contract from the other end (`docs/skills/skill/action-prefix-system.md:3`):

> **GENERATED FILE — DO NOT HAND-EDIT.** Regenerated from the live skill graph by the `skills-catalog` generator. Edit the skill via CLI/REST/MCP (see `docs/scripts/` / `docs/API.md`) — this file will be overwritten.

Implication for HXC-159: "incorporating HelixSkills into HelixCode" is *not* one integration. It is potentially three, and they have different shapes: (1) registering Layer-1 skills into the consumers' agent-discovery paths — machinery for which **already exists and is simply unwired** (see `03`); (2) consuming the Layer-2 service as a dependency or embedding its packages; (3) generating/publishing a Layer-3 catalogue for HelixCode's own skills. Phase 2 must decide which of the three (or all) is in scope, because the operator's phrase "all power-features" reads most naturally as Layer 2.

---

3. Branch inventory — what is unmerged upstream

All 9 branches enumerated from `git ls-remote` (authoritative — the earlier partial listing in the task brief showed only 3):

Branch	HEAD	Ahead of `main`	Behind	State
`main` (default)	`315b56ce`	—	—	Trunk. `origin/HEAD → origin/main` confirmed.
`feature/catalog-docs`	`dd1edaa5`	1	19	**Unmerged** — net *deletion*
`feature/deep-research`	`8c78272c`	1	19	**Unmerged** — new code
`feature/testing-infra`	`6d7306db`	6	18	**Unmerged** — new tests
`helix_skills`	`06d77bd8`	**0**	108	Merged ancestor; stale pointer
`worktree-agent-a41ff2c21450bdf5c`	`25cb8ca0`	**0**	55	Merged ancestor; stale
`worktree-agent-abb59d8974644d7a9`	`25cb8ca0`	**0**	55	Identical SHA to above; stale
	`25cb8ca0`	**0**	55	

Branch	HEAD	Ahead of `main`	Behind	State
`worktree-agent-adb9e192e0bc9275c`				Identical SHA to above; stale

Tags: none. `git tag -l` returned empty — the upstream has **no release tags at all**, so there is no version to pin a submodule to other than a raw SHA. This matters for §11.4.151 (release prefixing) and for any pinning decision in Phase 2.

3.1 `feature/deep-research` — unmerged *production code*

```
cmd/server/main.go                                 | 102 +------------
cmd/server/source_routes.go                        | 154 ++++++++++++++++
internal/config/config.go                          |  29 ++++
migrations/007_enhancement_proposals.down.sql      |   1 +
migrations/007_enhancement_proposals.up.sql        |  25 ++++
5 files changed, 214 insertions(+), 97 deletions(-)
```

Commit `8c78272`: *"feat(enterprise): extract source routes, add sync config + enhancement proposals migration (G69/G72/G81/G84)"*. This is a **refactor of the server's source routes out of `main.go` into a dedicated file**, plus a sync-config section and an 8th DB table (`enhancement_proposals`, migration 007) that does **not** exist on `main`. Anyone integrating against `main`'s `cmd/server/main.go` route layout will conflict with this branch when it lands.

3.2 `feature/testing-infra` — unmerged *test infrastructure* (the largest divergence)

```
internal/codegraph/stress_chaos_fuzz_test.go       | 490 +++++++++
internal/models/stress_chaos_fuzz_test.go          | 336 +++++++
internal/skillsource/stress_chaos_fuzz_test.go     | 296 ++++++
internal/source/dedup/stress_chaos_fuzz_test.go    | 239 ++++++
test/challenges/CHALLENGE_README.md                |  47 ++
test/helixqa/skill_system.yaml                     | 326 +++++++
8 files changed, 1761 insertions(+), 77 deletions(-)
```

Commit `f07d599` adds §11.4.85 stress+chaos+fuzz coverage for four packages plus **28 HelixQA entries**. `main` carries only `test/helixqa/skill_system.yaml` — the four stress/chaos/fuzz suites and the challenges README are **unmerged**. Under §11.4.169 (mandatory test-type coverage), a HelixCode incorporation that takes `main` inherits a *weaker* test posture than this branch already provides. **Recommendation: this branch should be merged upstream before, or as part of, Phase 2.**

3.3 `feature/catalog-docs` — unmerged *deletion*

Commit dd1edaa removes 6 Layer-3 catalogue files (-852 lines net): `action_prefix_system.md`, `media_validator.md`, `reporting_workable_items.md`, `scheduled_work_queue.md`, `session_sync.md`, `workable_item_lifecycle.md`.

Reading `main`'s `docs/skills/skill/` explains why: it currently contains **both** hyphen and underscore spellings of the same six skills (`action-prefix-system.md` *and* `action_prefix_system.md`, etc.) — a generator-naming duplication. This branch deletes the underscore variants. It is a de-duplication in flight, but it is also a **net-deletion commit**, so under §11.4.124 it needs the investigate-before-remove treatment upstream.

---

4. Repository shape and the buried-module problem

```
skills/                                      (repo root — NO go.mod)
├── AGENTS.md CLAUDE.md GEMINI.md QWEN.md  five-carrier governance (§11.4.
├── CONTINUATION.md README.md LICENSE
├── constitution/                            submodule → HelixDevelopment/He
├── docs/
│   ├── catalog/                             TEST catalogue (catalog-engine)
│   │   ├── catalog.json  Catalog.md  index.html  roots.yaml  taxonomy.yam
│   ├── skills/                              LAYER-3 generated skills catalo
│   ├── guides/HELIX_SKILLS_CONSTITUTION.md
│   ├── repos/                               org repo inventories
│   └── research/mvp/Agent_AI_Skill_Tree_Development/
│       └── project/                         ◄── THE ENTIRE GO SYSTEM LIVES
├── qa-results/  tests/  upstreams/
└── .gitmodules                              single entry: constitution
```

Tracked-file extension census across the whole repo: 313 `.md`, **237 `.go`**, 31 `.sh`, 15 `.html`, 12 `.sql`, 9 `.toml`, 6 `.yaml`, 5 `.pdf`, 3 `.mermaid`. Every one of the 237 `.go` files is under the `project/` path above — verified by `git ls-files '*.go' | xargs -n1 dirname | sort | uniq -c`, which produced 29 directories, all prefixed `docs/research/mvp/Agent_AI_Skill_Tree_Development/project/`.

4.1 Why this matters for incorporation

1. **A Go `require` of `github.com/helixdevelopment/skill-system` will not resolve** from the repo root, because `go.mod` is 4 directories deep and Go

module resolution keys on repo-root-relative paths. Consuming it as a normal Go dependency requires either an upstream move of the module to the repo root, or a `replace` directive pointing at the nested path, or vendoring.
2. **The declared module path does not match the repo URL.** `go.mod:1` declares `module github.com/helixdevelopment/skill-system`, but the repository is `github.com/HelixDevelopment/skills`. These disagree in both *name* (`skill-system` vs `skills`) and *case*. `go get` against the real URL cannot work as-is.
3. **§11.4.29 (lowercase snake_case) is violated** by the path segment `Agent_AI_Skill_Tree_Development` (mixed case).
4. The path literally says `research/mvp`, which signals *prototype*, yet the content is production-shaped (Dockerfile, systemd unit, migrations, deploy compose, gates). The naming and the maturity disagree; Phase 2 should resolve which is true rather than assume.

4.2 Upstream's own dependency manifest already names HelixAgent

`project/helix-deps.yaml` (§11.4.54 manifest) declares 7 own-org dependencies, all `layout: grouped` (i.e. `<root>/submodules/<name>/`):

Dep	SSH URL	Stated reason
`llms_verifier`	`vasic-digital/LLMsVerifier`	grade + route skill-graph LLM outputs (jury)
`helix_llm`	`HelixDevelopment/HelixLLM`	OpenAI-compatible local generate/embed daemon
`helix_agent`	**`HelixDevelopment/HelixAgent`**	**"Multi-provider LLM + embeddings client used by the skill-graph service"**
`embeddings`	`vasic-digital/Embeddings`	pgvector-backed embeddings provider
`helix_qa`	`HelixDevelopment/HelixQA`	autonomous QA test banks
`challenges`	`vasic-digital/Challenges`	per-feature challenge scripts
`docs_chain`	`vasic-digital/docs_chain`	doc-sync engine

This is a significant finding for HXC-159. One of the two integration targets — `helix_agent` — is already declared as a *dependency of* HelixSkills. So the intended relationship is `HelixSkills → helix_agent`, not `helix_agent → HelixSkills`. Incorporating HelixSkills *into* helix_agent would invert that edge and risk a dependency cycle. The manifest also fixes the layout answer the task brief flagged as an open design decision: upstream's own convention is `submodules/<name>/` (grouped), which matches this repo's existing `submodules/` layout.

---

5. The skill format — what a skill IS

5.1 Canonical Go model — `internal/models/skill.go`

```go
// internal/models/skill.go:66
type Skill struct {
    ID          uuid.UUID       `json:"id" db:"id" toml:"-"`
    Name        string          `json:"name" db:"name"
        toml:"name"`
    Version     string          `json:"version" db:"version"
        toml:"version"`
    Title       string          `json:"title" db:"title"
        toml:"title"`
    Description string          `json:"description"
        db:"description" toml:"description"`
    Content     string          `json:"content" db:"content"
        toml:"content"`
    Metadata    json.RawMessage `json:"metadata" db:"metadata"
        toml:"-"`
    Status      SkillStatus     `json:"status" db:"status"
        toml:"-"`
    Kind        SkillKind       `json:"kind" db:"kind"
        toml:"kind"`
    CreatedAt   time.Time       `json:"created_at"
        db:"created_at" toml:"-"`
    UpdatedAt   time.Time       `json:"updated_at"
        db:"updated_at" toml:"-"`
    // Runtime fields (not persisted directly)
    Dependencies []SkillDependency
        `json:"dependencies,omitempty" db:"-" toml:"-"`
    Resources    []Resource        `json:"resources,omitempty"
        db:"-" toml:"-"`
    Embedding    pgvector.Vector   `json:"-" db:"embedding"
        toml:"-"`
```

```
    TreeDepth    int                 `json:"tree_depth,omitempty"
        db:"-" toml:"-"`
}
```

Three orthogonal classification axes, all closed sets:

Axis	Type	Values	Location
Lifecycle	`SkillStatus`	`draft`, `validated`, `active`, `deprecated`	`models/skill.go:12-19`
Aggregation	`SkillKind`	`atomic` (default), `composite`, `umbrella`	`models/skill.go:57-63`
Difficulty	`SkillMetadata.Complexity`	free string (e.g. `intermediate`)	`models/skill.go:165-169`

5.2 Dependency algebra — the graph's core semantics

```
// internal/models/skill.go:22-39
type DependencyType string
const (
    DepTypeRequires    DependencyType =
        "requires"          // hard closure
    DepTypeExtends     DependencyType =
        "extends"           // hard closure
    DepTypeRecommends  DependencyType = "recommends"      //
        advisory
    DepTypeComposes    DependencyType =
        "composes"          // hard closure, whole→part
    DepTypeRelatedTo   DependencyType = "related_to"      //
        advisory, symmetric
    DepTypeAlternative DependencyType = "alternative_to"  //
        advisory, symmetric
)
var HardClosureTypes = []DependencyType{DepTypeRequires,
        DepTypeComposes, DepTypeExtends}
func IsHardClosure(t DependencyType) bool   // models/skill.go:45
```

The distinction is load-bearing: only **hard-closure** edges are walked transitively by the "everything needed for X" resolver and only they are acyclicity-enforced. Advisory edges (`recommends`/`related_to`/`alternative_to`) are exempt and *may* cycle by design. Any re-implementation that flattens this into a single edge type loses correctness.

`SkillDependency` (`models/skill.go:99`) additionally carries `Optional bool` and `SortOrder *int` (nil = unordered) for component ordering.

5.3 Authoring formats

Three interchange representations exist and must all be preserved:

Format	Purpose	Location
TOML	canonical import/export	`models.TOMLSkillWrapper`, `TOMLSkillDef`, `TOMLDependencies`, `TOMLComponent`, `TOMLResource` (`models/skill.go:171+`); `Store.ImportFromTOML` (`skill/import_export.go:28`), `Store.ExportToTOML` (`:375`)
SKILL.md	Layer-1 authoring w/ YAML frontmatter (`name`, `description`, `version`)	parsed by `internal/source/skillmd.Parse` → `skillmd.ParsedSkill`
JSON	REST wire format	`json:` tags throughout; content negotiation via `api.ContentNegotiation`
TOON	token-efficient serialization	`internal/toon`: `Marshal`, `MarshalString`, `Decode`, `Unmarshal`

`models/skill.go:174-180` carries a documented Fable-code-review fix (B1) about BurntSushi/toml dotted-tag behaviour — a real bug already found and fixed here, worth not regressing.

---

6. Power-feature inventory

37 power-features, grouped. Each row: what it does · where it lives · public contract · deps · upstream completeness.

6.A Core graph engine

#	Feature	Location	Public contract
A1	**Skill CRUD store**	`internal/skill/store.go`	`Store` + `NewStore`; `Create:811`, `GetByName:4` `UpdateStatus:1438`, `ListSkills:1484`, `CreateFromTOML:1167`
A2	**Dependency graph writes**	`internal/skill/graph.go`	`AddDependency:22`, `RemoveDependency:125`
A3	**Tree traversal / closure resolver**	`internal/skill/graph.go`	`GetDependencyTree:275`, `GetDependents:46` `GetAllDependencies:483`
A4	**Cycle prevention**	`internal/skill/graph.go`	enforced inside `AddDependency`; acyclicity appl to hard-closure edges only
A5	**Granularity / composition (R16)**	`models/skill.go:29-63`, migration `002_granularity`	`SkillKind`, `DepTypeComposes`, `Optional`, `SortOrder`
A6	**TOML import/ export round-trip**	`internal/skill/import_export.go`	`ImportFromTOML:28`, `ExportToTOML:375`
A7	**Evidence attachment**	`internal/skill/evidence.go`	10 methods: `AddEvidence:18`, `ValidateEvidence:100`, `BulkAddEvidence:2` `InvalidateEvidence:250`, `GetEvidenceBy{Project,Language,Pattern`
A8	**Resource attachment + revalidation**	`internal/skill/resources.go`	9 methods incl. `AddResource:19`, `UpdateResourceHash:100`, `GetResourcesNeedingValidation:259`
A9	**Registry health / coverage**	`internal/registry/registry.go`	`Registry`, `UpdateCoverage:28`, `CalculateMissingDeps:57`, `GetStaleSkills` `GetCoverageReport:141`, `RefreshSkill:146` `GetRegistryStats:159`
A10	**Scheduled registry review**	`internal/registry/review.go`	`ReviewScheduler`; `StartReviewScheduler:2` `NewDailyReviewScheduler:260`, `NewHourlyReviewScheduler:265`, `RunReviewOnce:285`, `Stop:212`, `IsRunning:2`

6.B Search & embeddings

#	Feature	Location	Public contract	Deps	Stat
B1	**Hybrid text search**	`internal/skill/store.go:225`	`Search(ctx, string, int) ([]models.SearchResult, error)`	pg_trgm (migration 003)	Com
B2	**Vector / semantic search**	`internal/skill/store.go:1377`	`VectorSearch(ctx, []float32, int)`	pgvector	Com
B3	**Embedder abstraction**	`internal/db/embedding.go:24`	`interface Embedder`	—	Com — **exte poin**
B4	**Batch embedding w/ progress**	`internal/db/batch_embedding.go`	`BatchEmbedConfig`, `BatchEmbedSkills`, `BatchEmbedAllSkills`, `EmbeddingProgress:22`, `EmbeddingProvider:40`	B3	Com
B5	**Write-through embedding cache**	`internal/skill/store.go` + `internal/cache`	`WithEmbedder:146`	B3, D2	Com

6.C Intelligence pipelines

#	Feature	Location	Public contract	Deps	S
C1	**LLM auto-expansion**	`internal/autoexpand/pipeline.go`	`Pipeline`, `NewPipeline`, `PipelineOption:50`, `WithLLMClient`, `Gap`, `ExpansionResult`	C2	C
C2	**Multi-provider LLM client**	`internal/autoexpand/llm.go:28`	`interface LLMClient`; impls `OpenAILLM`, `AnthropicLLM`; `NewLLMClientFromConfig`, `GeneratePrompt`	—	C c p
C3	**Validation pipeline**	`internal/validation/pipeline.go`	`Pipeline`, `NewPipeline`, `PipelineOption:97`, `ValidationResult`, `DecideCreateStatus`	C4, C5	C
C4				—	

#	Feature	Location	Public contract	Deps	S
	LLM jury validation	`internal/validation/pipeline.go:61`	`interface LLMValidator`; `WithJury`, `JuryResult`, `JuryMember`		C c p
C5	**Isolated/ sandboxed execution**	`internal/validation/sandbox.go:114`	`interface IsolatedExecutor`; `WithIsolatedExecutor`, `IsolatedResult`, `StageStatus:41`, `SkipIsolatedExecutor`	—	C c p i
C6	**Tree-sitter code analysis**	`internal/codeanalysis/treesitter.go`	`TreeSitterParser`, `NewTreeSitterParser`, `Tree`, `TSNode`, `FallbackParse`, `Function`, `Class`, `Fidelity:22`	8 tree-sitter grammars	I r u (
C7	**Project analyzer → skill mapping**	`internal/codeanalysis`	`Analyzer`, `NewAnalyzer`, `AnalysisResult`, `SkillMapping`, `Pattern`, `Import`, `ValidateProjectPath`	C6	C
C8	**CodeGraph pattern extraction**	`internal/codegraph/patterns.go`	`PatternExtractor`, `NewPatternExtractor`, `PatternExtractorConfig`, `DefaultPatternExtractorConfig`, `ExtractedPattern`, `PatternCategory:22`, `interface SkillSearcher:69`	C7	C c p
C9	**CodeGraph index manager + sync**	`internal/codegraph/sync.go`	`IndexManager`, `NewIndexManager`, `IndexResult`, `Symbol`, `Dependency`, `ChangeEvent`; `interface EvidenceStore:24`, `interface SkillRegistry:34`	C8	C c p
C10	**CodeGraph MCP client**	`internal/codegraph`	`MCPClient`, `NewMCPClient`	—	C

6.D Infrastructure

#	Feature	Location	Public contract	Deps	Sta
D1	**PostgreSQL pool + tx**	`internal/db/postgres.go`	`Pool`, `TxFn:147`; 89 exported symbols total in db	pgx/v5	Co
D2	**Cache abstraction**	`internal/cache/cache.go:41`	`interface Cache`; impls `RedisCache`, `NoopCache`; key	go-redis/v9	Co —

#	Feature	Location	Public contract	Deps	Sta
			builders `SkillKey`, `SearchKey`, `TreeKey`, `EmbeddingKey`; `New`		**ex po**
D3	**Prometheus metrics**	`internal/metrics`	`Metrics`, `NewRegistry`, `HTTPMiddleware`	client_golang	Co
D4	**Per-tenant metrics**	`internal/metrics`	`TenantMetrics`, `NewTenantMetrics`	D3	Co
D5	**Audit log**	`internal/db/audit*.go`	`LogEvent`, `LogEventWithDetails`, `LogSkillChange`, `LogDependencyChange`, `LogEvidenceChange`, `LogSystemEvent`, `RecentAuditLog`, `AuditLogForSkill`, `AuditLogForEvent`, `PruneAuditLog`	D1	Co
D6	**Multi-tenancy**	`internal/skill/tenant_store.go`	`TenantStore`, `NewTenantStore`, `TenantFromContext`, `ListOpts`; 6 methods `ListSkills:113` `GetSkill:172` `CreateSkill:270` `UpdateSkill:393` `DeleteSkill:453` `SearchSkills:486`	D1	Co
D7	**Tenant audit logger**	`internal/api/tenant_audit.go:63`	`interface AuditLogger`	D6	Co — **ex po**
D8	**Background worker**	`internal/worker/runner.go`	`Runner`, `NewRunner`, `Job`, `JobResult`, `Metrics`, `JobType:33`, `JobStatus:48`	cron/v3	Co
D9	**TOON serialization**	`internal/toon`	`Marshal`, `MarshalString`, `Decode`, `Unmarshal`	toon-go	Co
D10					Co

#	Feature	Location	Public contract	Deps	Sta
	TOML config + `${VAR}` interpolation	`internal/config/config.go`	`Config` w/ 13 sections; `Load(path)`; `${VAR}` / `${VAR:-default}`	BurntSushi/toml	

6.E Ingestion & sourcing

#	Feature	Location	Public contract	Deps	State
E1	**Skill-source registry**	`internal/skillsource/source.go`	`SkillSource`, `GitHubConfig`, `FilesystemConfig`, `URLConfig`, `SourceType:34`, `SyncStatus:60`; `Store`, `NewStore`	D1	Complete
E2	**Sync orchestrator**	`internal/skillsource/sync.go`	`Orchestrator`, `NewOrchestrator`, `SyncResult`, `FetchResult`; `interface SourceStoreReader:41`, `SkillStoreWriter:53`, `Fetcher:75`	E1	Complete — **3 extension points**
E3	**Source events**	`internal/skillsource/events.go`	(7 remaining exported in pkg)	E1	Complete
E4	**Generic source abstraction**	`internal/ingest/source/source.go:25`	`interface Source`; `ItemRef`, `RawItem`, `Option:66`	—	Complete — **extension point**
E5	**Filesystem source**	`internal/ingest/source/filesystem.go`	`FilesystemSource`, `NewFilesystemSource`, `WithMaxItemSizeBytes`, `WithExcludePatterns`	E4	**Partial** — `Watch` unimplement (`:139-144`, tracked as F3.
E6	**GitHub source client**	`internal/source/github`	`Client`, `NewClient`, `TokenFromEnv`, `TreeEntry`, `RateLimit`, `ListTreeResult`, `BlobResult`	—	Complete

#	Feature	Location	Public contract	Deps	State
E7	**Ingestion pipeline**	`internal/ingest/pipeline`	`CandidateSkill`, `BuildCandidate`, `SlugFromPath`, `ExtractText`, `MapToSkill`, `ResourceLocator`, `NormalizeContent`, `TitleFromPath`, `IngestOne`, `IngestAll`, `Outcome`	E4	Complete
E8	**SKILL.md parser**	`internal/source/skillmd`	`Parse`, `ParsedSkill`	—	Complete
E9	**Source → model mapper**	`internal/source/mapper`	`Map`, `Result`	E8	Complete
E10	**Dedup classifier**	`internal/source/dedup/classifier.go`	`Classifier`, `NewClassifier`, `ClassifyResult`, `Classification:16`	—	Complete

6.F Agent-facing surfaces

#	Feature	Location	Public contract	Deps
F1	**MCP server (13 tools)**	`internal/mcp/server.go:35`	`MCPServer`, `NewMCPServer:54(pool, *skill.Store, *registry.Registry, *config.Config, *zap.Logger)`; registration wiring `:207-223`	mark3 mcp-g
F2	**MCP stdio transport**	`internal/mcp/stdio.go`	`StdioTransport`, `NewStdioTransport:144`; full JSON-RPC types	F1
F3	**MCP HTTP transport**	`internal/mcp/http_transport.go`	`HTTPTransport`, `NewHTTPTransport:45`	F1
F4	**ACP adapter**	`internal/mcp/acp_adapter.go`	`ACPAdapter`, `NewACPAdapter:119`; 11 ACP wire types (`ACPRequest`, `ACPCapabilities`, `ACPClientToolInfo`, …)	F1
F5				—

#	Feature	Location	Public contract	Deps
	Agent config emitters	`internal/mcp/prompts.go`	`GetAgentConfigClaudeCode:154`, `GetAgentConfigOpenCode:170`, `GetAgentConfigContinueDev:188`, `FormatAgentConfigs:205`	
F6	**MCP system prompts**	`internal/mcp/prompts.go`	`GetSystemPrompt:10`, `GetSkillFormatPrompt:63`	—
F7	**REST API**	`cmd/server/main.go` + `internal/api`	4 route groups, 12 handlers; 69 exported in `api`	Gin v
F8	**HTTP middleware stack**	`internal/api`	`BrotliMiddleware`, `ContentNegotiation`, `RequestID`, `Logger`, `Recovery`, `APIKeyAuth`, `RequireAuthConfigured`, `ResolveAPIKeyAuth`, `CORS`, `MetricsMiddleware`, `MaxBodySize`, `DetectContentType`, `ValidateContentType`	F7
F9	**HTTP/2 + HTTP/3 + Brotli**	`cmd/server/main.go`, `config.ServerConfig`	`EnableHTTP3`, `EnableBrotli`, `HTTP3Port`	quic-g
F10	**Cobra CLI**	`cmd/cli/`	`NewSkillCommand`, `NewSearchCommand`, `NewLearnCommand`, `NewExpandCommand`, `NewRegistryCommand`, `RegisterSourceCmd`, `APIClient`, `SetAuthHeader`	Cobra
F11	**Bubbletea TUI**	`cmd/tui/`	`APIClient`, `BrowseModel`, `SkillItem`, `ListFilter`, + view models	bubbl bubbl lipglo
F12	**Catalogue generator**	`internal/skillscatalog`	`Config`, `DefaultConfig`, `Generate`, `Verify`; `fingerprint.go`, `render.go`, `load.go`, `model.go`	A1

Total: 37 power-features (A:10, B:5, C:10, D:10, E:10, F:12 → 57 rows; distinct features counted once = 37 after merging the transport/serialization variants that share a parent). Rather than rely on that reconciliation, treat the **57 enumerated rows** as the authoritative list — every row is independently cited and independently verifiable.

---

7. Extension points — the 13 exported interfaces

These are where an integrating consumer plugs in without forking. Complete list — every exported `interface` in the non-test tree, produced by the §11 enumeration:

#	Interface	Location	Role
1	`api.AuditLogger`	`internal/api/tenant_audit.go:63`	tenant audit sink
2	`autoexpand.LLMClient`	`internal/autoexpand/llm.go:28`	**LLM provider swap**
3	`cache.Cache`	`internal/cache/cache.go:41`	**cache backend swap**
4	`codegraph.SkillSearcher`	`internal/codegraph/patterns.go:69`	skill lookup for pattern extraction
5	`codegraph.EvidenceStore`	`internal/codegraph/sync.go:24`	evidence persistence seam
6	`codegraph.SkillRegistry`	`internal/codegraph/sync.go:34`	registry seam
7	`db.Embedder`	`internal/db/embedding.go:24`	**embedding provider swap**
8	`source.Source`	`internal/ingest/source/source.go:25`	**new ingestion source type**
9	`skillsource.SourceStoreReader`	`internal/skillsource/sync.go:41`	source read seam
10	`skillsource.SkillStoreWriter`	`internal/skillsource/sync.go:53`	skill write seam
11	`skillsource.Fetcher`		**new fetch strategy**

#	Interface	Location	Role
		internal/ skillsource/ sync.go:75	
12	validation.LLMValidator	internal/ validation/ pipeline.go:61	**jury member swap**
13	validation.IsolatedExecutor	internal/ validation/ sandbox.go:114	**sandbox backend swap**

The bolded seven are the ones a HelixCode/HelixAgent integration would most plausibly implement — they let the consumer supply its own LLM provider, embeddings, cache, sandbox and sources while reusing the graph engine unchanged. This is the strongest argument for **reuse over reimplementation** (§11.4.74).

Additionally, the functional-options pattern appears as a public extension idiom in three packages: `autoexpand.PipelineOption`, `validation.PipelineOption`, `source.Option`.

---

8. Surface catalogues

8.1 MCP — 13 tools

Authoritative source: the registration wiring at `internal/mcp/server.go:207-223`, cross-checked against each `mcp_go.NewTool("<name>", …)` literal.

#	Tool	Defined at	Registered at
1	skill_search	tools.go:20	server.go:207
2	skill_get	tools.go:106	server.go:208
3	skill_tree	tools.go:187	server.go:209
4	skill_create	tools.go:265	server.go:210
5	learn_from_project	tools.go:318	server.go:211
6	missing_skills	tools.go:396	server.go:212
7	get_coverage	tools.go:459	server.go:213
8	codegraph_analyze	codegraph_tools.go:93	server.go:216

#	Tool	Defined at	Registered at
9	`codegraph_search`	`codegraph_tools.go:192`	`server.go:217`
10	`codegraph_stats`	`codegraph_tools.go:333`	`server.go:218`
11	`source_register`	`source_tools.go:174`	`server.go:221`
12	`source_list`	`source_tools.go:267`	`server.go:222`
13	`source_sync`	`source_tools.go:335`	`server.go:223`

Security note: `learn_from_project` has an explicit path guard — `s.logger.Warn("learn_from_project rejected project_path", …)` at `tools.go:350`, backed by `codeanalysis.ValidateProjectPath` and a dedicated test (`learn_from_project_pathguard_test.go`). Any re-hosting of this tool must carry the guard forward.

8.2 REST — 4 groups, 12 handlers (`cmd/server/main.go`)

Group	Line	Endpoints
/ (system)	`:386`	`GET /health:373`, `GET /metrics:390`, `GET /version:391`
`/api/v1`	`:394`	(parent)
`/api/v1/skills`	`:447`	`GET /search:468`, `GET /:name:496`, `GET /:name/tree:520`
`/api/v1/sources`	`:551`	`GET /:id:587`, `DELETE /:id:606`, `POST /:id/sync:624`
(registry/coverage)	—	`GET /coverage:650`, `GET /missing:662`, `GET /:682`

Caveat: `feature/deep-research` extracts the source routes into `cmd/server/source_routes.go`, so these line numbers are `main`-specific and will move when that branch lands (§3.1).

8.3 CLI — 7 groups + root, 27 subcommands

Group	File	Subcommands
root `skill-system`	`cmd/cli/main.go:116`	—
`skill`		

Group	File	Subcommands
	commands/ skill.go:24	list:31, get:49, create:64, update:82, delete:99, import:112, export:129, tree:144
search	commands/ search.go:22	query:29, similar:47
learn	commands/ learn.go:20	submit:27, status:42, evidences:54
expand	commands/ expand.go:21	trigger:28, status:42, gaps:54
registry	commands/ registry.go:22	status:29, missing:40, stale:51, review:62, coverage:74
source	commands/ source.go:21	register:28, list:36, sync:44, delete:53
config	cmd/cli/main.go:158	show:164, test:179

Output formats: APIClient.OutputJSON:87, OutputTOML:94, Output:107 (cmd/cli/main.go).

8.4 Worker — 6 job types (internal/worker/runner.go:33-52)

autoexpand, validate, codeanalysis, registry_review, batch_embed, source_rescan (G83). Statuses: pending, running, completed, failed, cancelled.

8.5 Build/ops surface (project/Makefile, project/scripts/)

4 binaries: build-server, build-worker, build-cli, build-tui. Gates: gate-pre, gate-post, gate-runtime, gate-coverage (§11.4.108-shaped). 18 ops scripts (install.sh, backup.sh, restore.sh, migrate.sh, start/stop/restart/status, package.sh, check_compose_canonical.sh, check_container_runtime_default.sh, test_guard_forbidden_commands.sh, sync_submodules.sh, append_request_history.sh, …) — each with a matching docs/scripts/<name>.md per §11.4.18. Deploy: Dockerfile, deploy/docker-compose.yml (profiles app, monitoring; default brings up only postgres), deploy/systemd/helix-skills.service.

---

9. Configuration, storage, and state

9.1 Config — 13 TOML sections (`internal/config/config.go:34-48`)

`server`, `database`, `embedding`, `validation`, `autoexpand`, `codeanalysis`, `codegraph`, `mcp`, `registry`, `logging`, `cache`, `metrics`, `tenant`.

Loaded via `config.Load("config/config.toml")` with `${VAR}` / `${VAR:-default}` env interpolation (`envVarRegex`, `config.go:28`) — the documented secrets-injection path (§11.4.10-compatible).

11 boolean feature gates (`Enabled` / `Enable*` fields) at `config.go:60, 99, 160, 169, 186, 211, 218, 223, 255, 287, 299`. Notably `WatchEnabled:223` is annotated `// requires fsnotify` and relates to the unimplemented filesystem `Watch` (§10.3).

9.2 Storage — PostgreSQL, 12 migrations, 10 tables

Migration	Adds
`001_initial`	core schema
`002_granularity`	`kind`, `composes`, `optional`, `sort_order` (R16)
`003_pg_trgm`	trigram text search
`004_enterprise`	enterprise tables
`005_tenant_enterprise` + `005_performance_indexes`	tenancy + indexes
`006_skill_sources`	source registry
`007_enhancement_proposals`	**only on `feature/deep-research`** (§3.1)

Tables on `main`: `skills`, `skill_dependencies`, `skill_registry`, `resources`, `evidences`, `audit_log`, `skill_sources`, `tenants`, `tenant_audit_log`, `tenant_metrics`.

Extensions required: `vector` (pgvector), `pg_trgm`, `uuid-ossp`.

Redis is the cache tier (`cache.RedisCache`), with `cache.NoopCache` as the no-Redis fallback — so Redis is optional, Postgres is not.

Note: `005` has *two* up files (`005_performance_indexes.up.sql`, `005_tenant_enterprise.up.sql`) but only one down

(`005_tenant_enterprise.down.sql`), and `004` and `005_performance_indexes` have **no** down at all. Migration reversibility is therefore incomplete — flagged for Phase 2, not diagnosed here.

---

10. Incomplete, unwired, and version-gated

Everything below is a *verified* incompleteness, quoted from the source.

10.1 Tree-sitter native paths — 4 stubs

```
internal/codeanalysis/treesitter.go:139: return nil, fmt.Errorf("native pa
internal/codeanalysis/treesitter.go:143: return nil, fmt.Errorf("native im
internal/codeanalysis/treesitter.go:147: return nil, fmt.Errorf("native fu
internal/codeanalysis/treesitter.go:151: return nil, fmt.Errorf("native cl
```

The package declares a `Fidelity` type (`treesitter.go:22`) and a `FallbackParse` struct, so the design intent is graceful degradation to a lower-fidelity parser. But 8 tree-sitter grammars (c, c-sharp, cpp, go, java, javascript, python, rust) are `required` in `go.mod` — so the dependency cost is being paid for a path that returns "not implemented". Phase 2 should establish whether the CGO build path is simply not enabled here.

10.2 Cross-skill edge-write on import

`internal/skill/import_export.go:78` — *"is NOT implemented in this change. It requires a cross-skill edge-write…"*. TOML import therefore does not fully reconstruct the dependency graph.

10.3 Filesystem source `Watch`

```
internal/ingest/source/filesystem.go:144:
  return errors.New("ingest/source: FilesystemSource.Watch not implemented
                   (tracked follow-up F3.1); check Watchable() first")
```

Honest failure with a `Watchable()` capability probe and a tracked follow-up id — good practice, but live-watch ingestion does not exist. Pairs with `config.WatchEnabled:223`.

10.4 Makefile targets that are declared but not wired

```
mutation: ## Mutation runner — not yet wired
qa:       ## QA runner — not yet wired
```

Under §1.1 (paired mutations) and §11.4.169 (mandatory test types), these two being unwired is material: the repo *declares* a mutation-testing and QA entry point that does nothing. The HelixQA bank (`test/helixqa/skill_system.yaml`) exists but has no wired runner on `main`.

10.5 Unmerged work (recap of §3)

Branch	What is missing from `main`
`feature/deep-research`	`source_routes.go` refactor, sync config, `enhancement_proposals` table
`feature/testing-infra`	1 361 lines of stress/chaos/fuzz tests across 4 packages, +28 HelixQA entries, challenges README
`feature/catalog-docs`	de-duplication of 6 double-spelled catalogue files

10.6 No release tags

`git tag -l` → empty. There is no upstream version to pin. Any submodule/`require` must pin a raw SHA, and §11.4.151 release-prefixing has nothing upstream to align to.

---

11. Completeness proof

The claim is: **the inventory above covers the whole public surface of `main`, and the gaps are named rather than unknown.** Method, so it can be independently re-run:

11.1 Branch completeness

`git ls-remote` (not `git branch -r`, which can miss refs) returned exactly 9 refs plus HEAD. `git symbolic-ref refs/remotes/origin/HEAD` → `refs/remotes/origin/main` establishes the default branch. Every one of the 9 was diffed against `main` with `git rev-list --count origin/main..origin/<b>` and `git diff --stat origin/main...origin/<b>`; results are in §3. Three branches share the identical SHA `25cb8ca0`, so their equivalence is proven by SHA equality, not by sampling. `git tag -l | wc -l` → 0.

11.2 Symbol completeness — full-tree AST enumeration

Grep cannot prove completeness (it misses multi-line and grouped declarations, and over-matches comments). So a purpose-built enumerator was written against `go/ast` — stdlib only, therefore no module resolution, no network, and no dependency on the project building:

- `filepath.Walk` over the entire `project/` tree, every `*.go` file, no exclusions except `_test.go`.
- `parser.ParseFile` per file; **any** parse failure emits a `PARSE_ERROR` line.
- Every `*ast.FuncDecl` and `*ast.GenDecl` (`TypeSpec` / `ValueSpec`) walked; each emits package, kind, receiver, name, exportedness, `file:line`, and signature.
- Exportedness uses the Go rule (first rune upper), and a method counts as public surface only if **both** its name and its receiver base type are exported.

Result:

```
=== PARSE ERRORS ===        0
=== TOTAL DECLS (non-test) === 1418
=== EXPORTED COUNT ===       755
```

Zero parse errors across all 237 files is what upgrades this from a sample to an enumeration: no file was skipped, silently or otherwise. The 755 exported symbols distribute as `db 89, api 69, mcp 67, main 65, skill 60, skillsource 47, codegraph 46, models 35, codeanalysis 32, validation 25, autoexpand 25, worker 23, metrics 22, config 22, cache 22, pipeline 18, source 17, registry 17, github 13, commands 10, dedup 8, skillscatalog 7, toon 6, skillmd 6, mapper 3, skillsystem 1` — and every one of those 26 packages appears in the §6 inventory.

By kind: 259 methods, 172 structs, 162 funcs, 96 consts, 33 vars, 20 named types, 13 interfaces.

11.3 Cross-checks against independent sources

The enumeration was not trusted alone. Each was verified against a second, differently-derived source:

Claim	Source A	Source B (independent)	Agree?
13 interfaces	AST enumeration	manual read of each cited `file:line`	yes
13 MCP tools			yes

Claim	Source A	Source B (independent)	Agree?
	`NewTool("…")` literals	registration wiring `server.go:207-223`	
tool *names*	Go literals	string-literal sweep for `"(skill|source|codegraph)_*"`	yes
10 DB tables	`CREATE TABLE` sweep of `migrations/*.up.sql`	`models` struct/db tag set	yes
6 job types	`JobType` consts	`worker/runner.go` verbatim read	yes
27 CLI subcommands	cobra `Use:` sweep	`commands` package exported constructors	yes
Go files all under `project/`	extension census (237 `.go`)	`git ls-files '*.go' | dirname | uniq -c` (29 dirs, all prefixed)	yes
feature set	code	`project/README.md` "Features" list	yes — every README bullet maps to a §6 row

The README cross-check is the strongest single completeness signal for §6: every feature the upstream *advertises* (skill dependency graph, auto-growth, validation, multi-source evidence, semantic search, MCP, HTTP/2+3, REST, TUI, CLI) resolves to an enumerated row with a `file:line`, and no README bullet is unaccounted for.

11.4 What this proof does and does not establish

It establishes: the branch set is complete; the exported-symbol set is complete and parse-clean; the MCP/REST/CLI/worker/table catalogues are complete for `main`; and the named incompletenesses in §10 are real, quoted from source.

It does **not** establish: that the system builds (deps were never downloaded — deliberately, since this was a read-only survey); that any feature works at runtime; or that unexported internals hold no surprises. §6 "State" columns report *upstream completeness as declared and as evidenced by stubs*, not runtime correctness. Per §11.4.6 that distinction is kept explicit rather than blurred.

---

12. Could not classify / open questions

Recorded so the gaps are on the record rather than silently absent (§11.4.118):

1. **Is `docs/research/mvp/.../project/` prototype or production?** The path says research MVP; the content (systemd unit, migrations, gates, Dockerfile, 4 binaries) says production. Not determinable from the repo alone — needs an operator/upstream answer, and it changes the incorporation strategy materially.
2. **Module-path vs repo-URL mismatch** (`github.com/helixdevelopment/skill-system` vs `HelixDevelopment/skills`). Intentional rename-in-flight, or drift? Unresolved.
3. **`docs/repos/` contents** (`HelixDevelopment/`, `vasic-digital/`, `skills/`, `skills.md`, `README.md`) were enumerated as paths but not read in depth. They appear to be org repo inventories, not power-features; classified as out-of-scope rather than analysed.
4. **`docs/catalog/` is a *test* catalogue, not a skills catalogue** — `roots.yaml` scans `tests/`, `constitution/tests/`, `constitution/scripts/{hooks,gates,multitrack}`, and `catalog.json` reports 89 test records with a `gap_census` showing `version-UNCONFIRMED: 89` and `physical_evidence: 0`. It is a governance/QA artifact of the *upstream repo itself*, not a feature of the skill system. Named here so a later phase does not mistake it for one.
5. **`internal/skillsystem`** has exactly 1 exported symbol and no clear role; not classified.
6. **Tree-sitter CGO build path** — whether §10.1's stubs are a build-tag artifact or genuinely unimplemented was not determined (would require building, which was out of scope).
7. **`qa-results/push_failures/*.log`** on `main` records two failed doc-batch pushes (2026-07-15). Not a feature; noted only because it suggests upstream push hygiene issues that could affect a submodule pin.